Reflection on Advanced Topics

Gulce Ozhan (2457288) and Oney Khambhadia (2422301)

During our Minesweeper CBL project, we applied two advanced topics, *namely Java Properties* for external configuration and *Git* for version control and collaboration. Both played an important role in the development process and helped us gain practical experience with software engineering techniques beyond basic Java programming.

1. Using Java Properties for Game Configuration

We used a `config.properties` file to separate the game's configuration parameters from the Java source code. This file contains settings such as the initial difficulty level, the number of mines, the dimensions of the minefield for each difficulty, and the size of individual cells.

In `MainController` class, we implemented the `readProperties()` method using Java's `Properties` class to load these settings when the game starts. The values are then used to initialize the game according to the selected difficulty level. This allowed us to modify the game's behaviour without changing or recompiling the Java code. For example, we could adjust the number of mines or dimensions of the board simply by editing the properties file.

One challenge was ensuring that the configuration values were correctly read and converted into the required data types, particularly integers. We also had to consider what would happen if the properties file could not be accessed. To address this, we provided default values in the controller and handled file-reading errors using a `try-catch` block.

Another challenge was, when executable JAR file was produced, we noticed it was no longer possible to access and change the config inside the JAR. We explored ways for externalization and implemented it. Now, if another `config.properties` file is put in the same directory as the JAR file, then the internal config is overridden with the one provided externally.

Through this experience, we learned the importance of separating configuration from application logic. This approach made our game more flexible, easier to maintain, and simpler to customize. Looking back, we could further improve our implementation by validating configuration values and handling invalid or missing properties more explicitly.

2. Using Git for Version Control and Collaboration

Since we worked as a team of two, Git was essential for organizing our development activities. We used GitHub as our shared repository and Git to track changes, manage separate development branches, and integrate our individual contributions.

We initially established the project structure and `README` before starting to develop different features in parallel. As shown in our Git history, we created *feature* branches for tasks such as mine placement, board mouse actions, and the top control panel. These branches allowed us to work independently without immediately affecting the main branch. Once features were

completed, we integrated them through pull requests (seen as pull requests #1, #2, and #3 in Git history). For small updates (like documentation, comments, minor refactoring), we directly pushed the changes on the *main* branch to avoid the hassle of branching and merging.

A particularly useful aspect of Git was its ability to preserve the history of our work. Each *commit* recorded a specific change, making it easier to understand how the project evolved and identify which features had been implemented. We also gained experience with *merging* branches and keeping our local work synchronized with the shared repository.

One of the challenges was coordinating changes between branches and ensuring that newly integrated features worked correctly together. This taught us that version control is not only about saving code, but also about communication, coordination, deconfliction and managing dependencies between team members.

Overall Reflection

Overall, these two advanced topics improved our development process and the resulting product. Java Properties taught us how to make software configurable without modifying its implementation, while Git enabled us to collaborate effectively and maintain a structured development history. Most importantly, we gained hands-on experience with practices commonly used in industry scale software projects. We believe these skills will be valuable in future team-based programming assignments and professional software development.